

AI 구매 Agent 상품 수집 시스템

소프트웨어 아키텍처 설계서

1. 문서 개요

1.1 목적

본 문서는 AI 구매 Agent에서 사용하는 상품 데이터 수집 시스템의 소프트웨어 아키텍처를 정의한다.

상품 수집 시스템은 여러 쇼핑몰에서 상품명, 가격, 옵션, 재고, 배송, 리뷰 등의 정보를 수집하고, 사이트별로 다른 데이터 구조를 공통 상품 모델로 변환하여 저장한다.

본 문서에서는 다음 사항을 정의한다.

- 시스템 구성 요소
- 컴포넌트별 책임
- 상품 수집 처리 흐름
- HTML, JSON API, 브라우저 기반 수집 방식
- 메시지 큐 및 저장소 구조
- 실패 처리와 재시도 정책
- 확장 및 운영 방식

2. 시스템 목표

2.1 기능 목표

시스템은 다음 기능을 제공해야 한다.

1. 수집 대상 쇼핑몰과 상품 URL을 등록할 수 있어야 한다.
2. 사이트별로 적절한 상품 수집 방식을 선택할 수 있어야 한다.
3. HTML, JSON API, 브라우저 렌더링 결과를 통해 데이터를 수집할 수 있어야 한다.
4. 사이트별 상품 데이터를 공통 데이터 구조로 변환할 수 있어야 한다.
5. 상품 데이터의 유효성을 검사할 수 있어야 한다.
6. 동일 상품과 동일 수집 요청의 중복 처리를 방지해야 한다.
7. 가격과 재고의 변경 이력을 저장해야 한다.
8. 실패한 작업을 재시도할 수 있어야 한다.
9. AI 구매 Agent가 상품 데이터를 조회할 수 있어야 한다.

10. 결제 직전에 최신 가격과 재고를 다시 검증할 수 있어야 한다.

2.2 비기능 목표

- 확장성: 수집 사이트 및 워커 수를 추가할 수 있어야 한다.
- 안정성: 일부 사이트나 워커 장애가 전체 시스템에 영향을 주지 않아야 한다.
- 유지보수성: 사이트별 크롤링 로직을 독립적으로 수정할 수 있어야 한다.
- 추적성: 데이터가 언제, 어디에서, 어떤 방식으로 수집됐는지 확인할 수 있어야 한다.
- 예절성: 대상 사이트에 과도한 요청을 보내지 않아야 한다.
- 신선도: 가격과 재고처럼 자주 변경되는 정보는 주기적으로 갱신해야 한다.

3. 시스템 범위

3.1 포함 범위

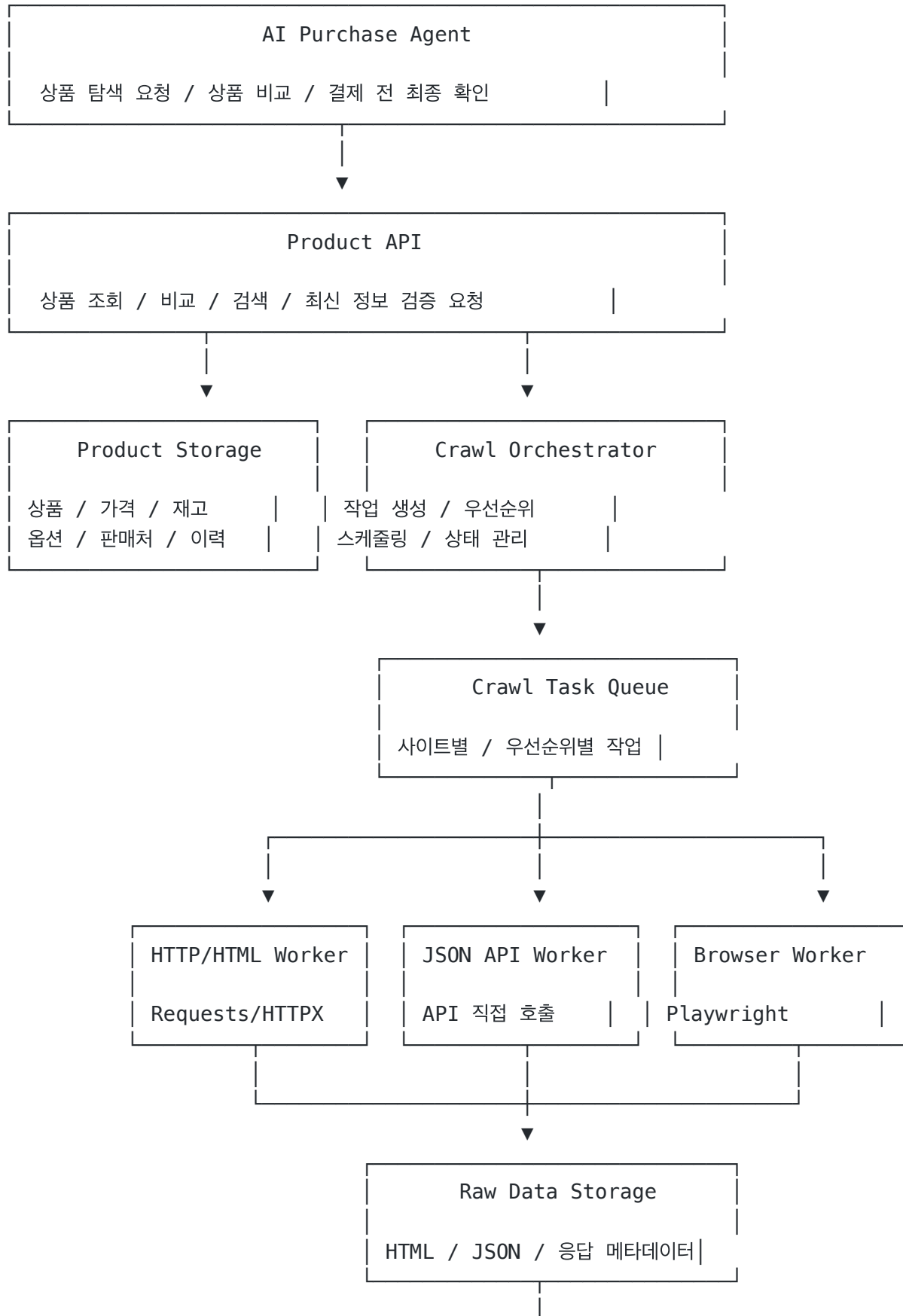
- 쇼핑몰 사이트 등록
- 수집 대상 URL 등록
- 상품 목록 수집
- 상품 상세 정보 수집
- HTML 파싱
- 내부 JSON API 호출
- Headless Browser 기반 수집
- 데이터 정규화
- 데이터 유효성 검증
- 중복 상품 판별
- 상품 및 수집 이력 저장
- 가격·재고 재수집
- AI Agent 조회 API
- 결제 전 상품 정보 재검증

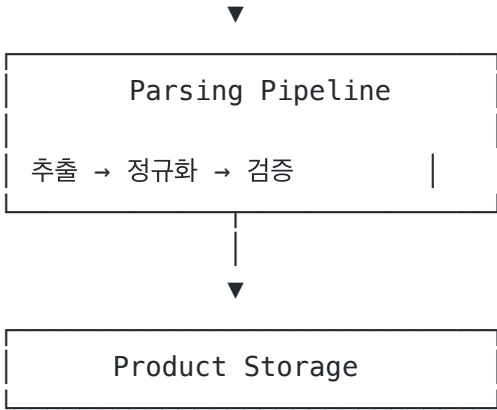
3.2 제외 범위

- 실제 카드 결제 처리
- 쇼핑몰 회원 계정 생성
- CAPTCHA 우회
- 접근 제한 우회
- 쇼핑몰의 보안 정책 회피
- 비공개 개인정보 수집

- 사용자 계정 정보 저장

4. 상위 수준 아키텍처





5. 핵심 컴포넌트

5.1 Crawl Orchestrator

역할

Crawl Orchestrator는 전체 수집 작업을 관리하는 중앙 제어 컴포넌트다.

주요 책임

- 수집 작업 생성
- 사이트별 수집 주기 관리
- 우선순위 계산
- 수집 작업 큐 등록
- 작업 상태 관리
- 실패 작업 재처리
- 중복 작업 방지
- 결제 전 긴급 재수집 요청 처리

입력

```
{
  "siteId": "SHOP_A",
  "targetType": "PRODUCT_DETAIL",
  "targetUrl": "https://shop-a.com/products/1234",
  "priority": "HIGH",
  "triggerType": "AGENT_REQUEST"
}
```

출력

Crawl Task Queue에 수집 작업을 전달한다.

```
{
  "taskId": "CRAWL_20260725_000001",
  "siteId": "SHOP_A",
  "targetUrl": "https://shop-a.com/products/1234",
  "collectorType": "JSON_API",
  "priority": 10,
  "retryCount": 0
}
```

5.2 Site Registry

사이트별 수집 설정과 Adapter 정보를 관리한다.

관리 항목

- 사이트 ID
- 도메인
- 수집 허용 여부
- 기본 수집 방식
- 요청 간격
- 동시 요청 수
- robots.txt 정책
- User-Agent
- Timeout
- Retry 횟수
- Adapter 버전
- 인증 필요 여부

설정 예시

```
sites:
  SHOP_A:
    domain: shop-a.com
    enabled: true
    collectorType: JSON_API
    requestIntervalMs: 1500
    maxConcurrency: 2
    timeoutSeconds: 10
    maxRetryCount: 3
    adapter: shop_a
```

adapterVersion: 1.2.0

```
SHOP_B:
  domain: shop-b.com
  enabled: true
  collectorType: BROWSER
  requestIntervalMs: 3000
  maxConcurrency: 1
  timeoutSeconds: 30
  maxRetryCount: 2
  adapter: shop_b
  adapterVersion: 1.0.0
```

5.3 Crawl Task Queue

수집 작업을 워커에 비동기적으로 전달한다.

큐 분리 기준

```
crawl.high
crawl.normal
crawl.low
crawl.browser
crawl.retry
crawl.dead-letter
```

우선순위 예시

우선순위	작업
긴급	결제 직전 가격·재고 확인
높음	사용자가 현재 검색한 상품
보통	인기 상품 주기적 갱신
낮음	전체 상품 동기화
재처리	일시적 실패 작업

사이트별 요청 속도를 제한하기 위해 siteId 또는 host 기준으로 작업을 분배한다.

5.4 Collection Strategy Resolver

대상 사이트와 URL을 분석해 어떤 수집 방식을 사용할지 결정한다.

수집 방식 선택 순서

```

공식 API
  ↓ 사용 불가
내부 JSON API
  ↓ 사용 불가
정적 HTML 파싱
  ↓ 데이터 부족
Headless Browser
  
```

선택 결과

```

{
  "siteId": "SHOP_A",
  "strategy": "JSON_API",
  "adapter": "shop_a",
  "reason": "상품 상세 API에서 가격과 재고 데이터 제공"
}
  
```

수집 방식을 런타임에 자동 탐지하기보다는 초기에는 사이트 분석 과정에서 결정하고, Site Registry에 설정하는 방식을 사용한다.

자동 탐지는 예외 상황에서만 보조적으로 사용한다.

5.5 HTTP/HTML Worker

HTML을 직접 내려받아 처리하는 워커다.

처리 과정

```

작업 수신
  → robots.txt 정책 확인
  → Rate Limit 확인
  → HTTP 요청
  → 응답 검증
  → 원본 HTML 저장
  → Parser 작업 생성
  
```

사용 기술 예시

- Python

- HTTPX
- aiohttp
- Requests
- Scrapy

주요 처리 항목

- Redirect 처리
- 압축 응답 처리
- 문자 인코딩 확인
- Timeout 처리
- HTTP 상태 코드 처리
- ETag 및 Last-Modified 처리
- 쿠키 및 세션 처리

5.6 JSON API Worker

브라우저 Network 탭에서 확인한 상품 API를 직접 호출하는 워커다.

역할

- 상품 목록 API 호출
- 상품 상세 API 호출
- 옵션 API 호출
- 재고 API 호출
- 리뷰 API 호출
- 페이지네이션 처리

요청 정의 예시

```
productDetail:
  method: GET
  path: /api/v1/products/{productId}
  headers:
    Accept: application/json
  responseMapping:
    name: data.productName
    price: data.salePrice
    stock: data.stockStatus
```

주의사항

내부 API는 쇼핑몰에서 외부 사용을 보장하는 공개 API가 아닐 수 있으므로, URL이나 파라미터가 변경될 수 있다.

따라서 API 호출 코드는 공통 Worker에 직접 구현하지 않고 사이트별 Adapter에 위치시킨다.

5.7 Browser Worker

JavaScript 실행이나 사용자 동작이 필요한 페이지를 수집한다.

사용 조건

- 초기 HTML에 상품 정보가 없는 경우
- API 요청에 브라우저 세션이 필요한 경우
- 옵션 선택 후 가격이 변경되는 경우
- 무한 스크롤이 필요한 경우
- 특정 버튼을 클릭해야 데이터가 표시되는 경우

사용 기술

- Playwright
- Chromium
- Browser Context Pool

처리 과정

브라우저 컨텍스트 확보

- 페이지 접속
- 렌더링 완료 대기
- 필요한 사용자 동작 수행
- DOM 또는 네트워크 응답 추출
- 원본 데이터 저장
- 컨텍스트 반환

리소스 관리

브라우저 프로세스를 요청마다 새로 생성하지 않는다.

Browser Process

```
├─ Context 1
│   ├── Page 1
│   └── Page 2
├─ Context 2
└─ Context 3
```

사이트 또는 세션 단위로 Browser Context를 분리하고, 일정 횟수 처리 후 폐기한다.

5.8 Site Adapter

사이트별 차이를 격리하는 핵심 컴포넌트다.

인터페이스 예시

```
from abc import ABC, abstractmethod
from typing import Any

class SiteAdapter(ABC):

    @abstractmethod
    async def collect_product_list(
        self,
        request: dict[str, Any],
    ) -> dict[str, Any]:
        pass

    @abstractmethod
    async def collect_product_detail(
        self,
        request: dict[str, Any],
    ) -> dict[str, Any]:
        pass

    @abstractmethod
    def parse_product(
        self,
        raw_data: dict[str, Any],
    ) -> dict[str, Any]:
        pass

    @abstractmethod
    def extract_next_targets(
        self,
        raw_data: dict[str, Any],
    ) -> list[dict[str, Any]]:
        pass
```

디렉터리 구조 예시

```
src/
├── core/
│   └── orchestrator/
```

```
|   |   | queue/
|   |   | scheduler/
|   |   | rate_limiter/
|   |   | storage/
|
|   | collectors/
|   | | http_collector.py
|   | | json_api_collector.py
|   | | browser_collector.py
|
|   | adapters/
|   | | shop_a/
|   | | | adapter.py
|   | | | selectors.py
|   | | | api_schema.py
|   | | | mapper.py
|   | |
|   | | shop_b/
|   | | | adapter.py
|   | | | selectors.py
|   | | | browser_actions.py
|   | | | mapper.py
|   | |
|   | | base.py
|
|   | pipeline/
|   | | parser.py
|   | | normalizer.py
|   | | validator.py
|   | | deduplicator.py
|
|   | domain/
|   | | product.py
|   | | crawl_task.py
|   | | crawl_result.py
```

5.9 Raw Data Storage

수집된 원본 응답을 저장한다.

저장 대상

- HTML 원문
- JSON 원문
- 렌더링된 HTML
- 요청 URL
- 요청 Header

- 응답 Header
- HTTP 상태 코드
- 수집 시각
- 수집 방식
- Adapter 버전
- Parser 버전

저장 목적

- 파서 오류 재현
- 사이트 구조 변경 분석
- 데이터 검증
- 파서 수정 후 재처리
- 수집 결과 추적

저장소 예시

- S3 또는 호환 Object Storage
- MinIO
- 로컬 파일 시스템은 개발 환경에서만 사용

경로 예시

```
raw/  
└─ SHOP_A/  
    └─ 2026/  
        └─ 07/  
            └─ 25/  
                └─ CRAWL_20260725_000001.json
```

5.10 Parsing Pipeline

원본 데이터를 상품 데이터로 변환하는 파이프라인이다.

```
Raw Data  
  ↓  
Extractor  
  ↓  
Normalizer  
  ↓  
Validator  
  ↓
```

Deduplicator



Product Storage

Extractor

사이트 응답에서 필요한 필드를 추출한다.

```
{
  "title": "무선 헤드폰",
  "salePrice": "279,000원",
  "stockText": "구매 가능"
}
```

Normalizer

사이트별 표현을 공통 데이터 형식으로 변환한다.

```
{
  "productName": "무선 헤드폰",
  "salePrice": 279000,
  "currency": "KRW",
  "stockStatus": "IN_STOCK"
}
```

Validator

다음 항목을 확인한다.

- 상품명 존재
- 가격 형식
- 가격 범위
- 상품 URL 형식
- 판매 상태
- 통화 단위
- 필수 옵션
- 데이터 수집 시각

Deduplicator

다음 값을 이용해 중복 상품을 판별한다.

- 사이트 상품 ID
- 모델 번호

- 브랜드
- 표준 상품 코드
- 정규화된 상품명
- 상품 이미지 해시

6. 도메인 모델

6.1 Product

```
{
  "productId": "PRD_000001",
  "canonicalProductId": "CP_000001",
  "productName": "Example Wireless Headphone",
  "brand": "Example",
  "modelNumber": "EX-1000",
  "categoryId": "CAT_AUDIO_HEADPHONE",
  "attributes": {
    "color": "black",
    "connectivity": "bluetooth"
  }
}
```

6.2 ProductOffer

동일 상품을 판매하는 판매처별 정보를 관리한다.

```
{
  "offerId": "OFF_000001",
  "productId": "PRD_000001",
  "siteId": "SHOP_A",
  "sourceProductId": "123456",
  "sellerId": "SELLER_001",
  "productUrl": "https://shop-a.com/products/123456",
  "originalPrice": 329000,
  "salePrice": 279000,
  "currency": "KRW",
  "stockStatus": "IN_STOCK",
  "shippingFee": 0,
  "collectedAt": "2026-07-25T17:00:00+09:00"
}
```

상품 자체와 판매 정보를 분리해야 여러 쇼핑몰의 동일 상품을 비교할 수 있다.

Canonical Product

- └ Shop A Offer
- └ Shop B Offer
- └ Shop C Offer

6.3 CrawlTask

```
{
  "taskId": "CRAWL_20260725_000001",
  "siteId": "SHOP_A",
  "targetType": "PRODUCT_DETAIL",
  "targetUrl": "https://shop-a.com/products/123456",
  "status": "PENDING",
  "priority": 10,
  "scheduledAt": "2026-07-25T17:00:00+09:00",
  "startedAt": null,
  "completedAt": null,
  "retryCount": 0
}
```

6.4 CrawlHistory

```
{
  "crawlHistoryId": "HIS_000001",
  "taskId": "CRAWL_20260725_000001",
  "collectorType": "JSON_API",
  "httpStatus": 200,
  "resultStatus": "SUCCESS",
  "rawDataLocation": "s3://crawler-raw/SHOP_A/...",
  "adapterVersion": "1.2.0",
  "parserVersion": "1.1.0",
  "durationMs": 532
}
```

7. 데이터베이스 설계

주요 테이블

```
site
site_collection_policy
crawl_target
crawl_task
```

```

crawl_history
raw_data_metadata
product
product_offer
product_option
price_history
stock_history
seller
crawl_failure

```

관계

```

site
├─ site_collection_policy
├─ crawl_target
└─ crawl_task
    └─ crawl_history
        └─ crawl_failure

```

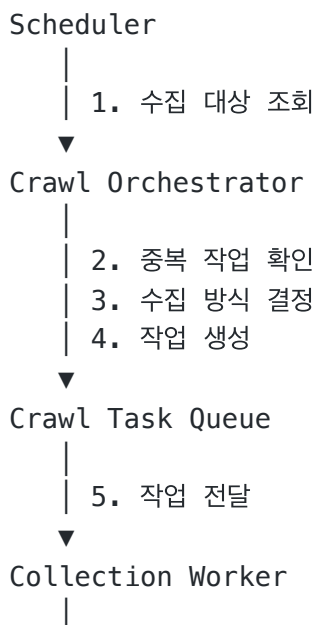
```

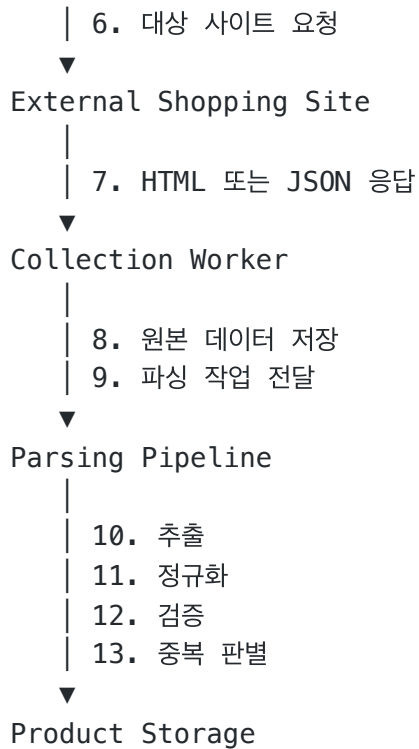
product
├─ product_offer
│   └─ product_option
│       └─ price_history
│           └─ stock_history
└─ canonical_product

```

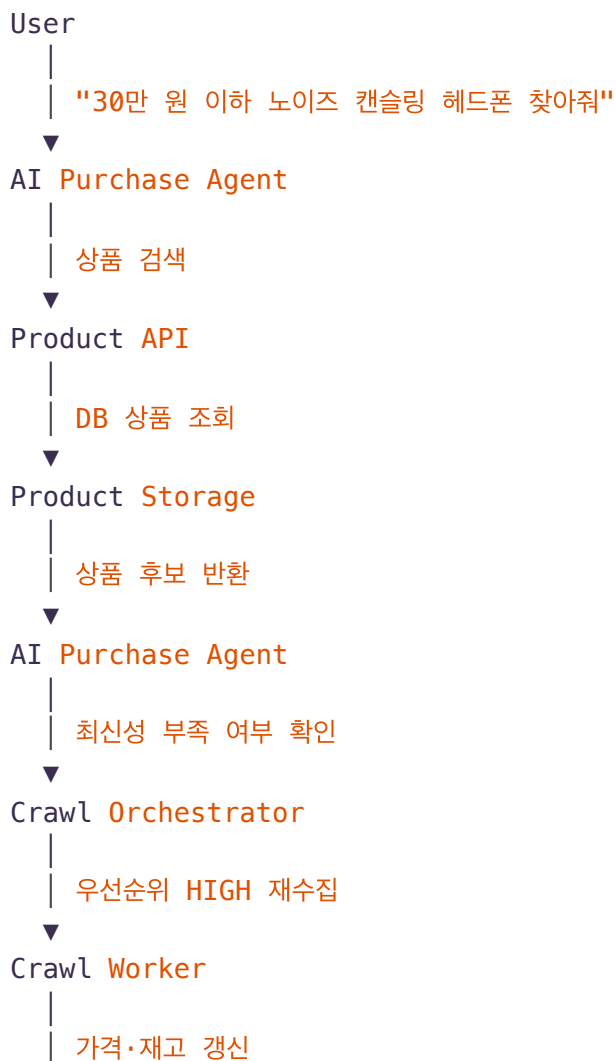
8. 주요 처리 시퀀스

8.1 상품 상세 수집





8.2 AI Agent 상품 요청



▼
AI Purchase Agent

8.3 결제 직전 재검증



9. 중복 처리

9.1 작업 중복

동일 사이트와 동일 URL에 대한 작업이 동시에 여러 개 생성되지 않도록 한다.

```
deduplicationKey =
SHA-256(siteId + targetType + normalizedUrl)
```

Redis를 사용하는 경우 다음과 같은 Key를 사용할 수 있다.

```
crawl:lock:SHOP_A:PRODUCT_DETAIL:123456
```

TTL은 예상 최대 수집 시간보다 길게 설정한다.

9.2 콘텐츠 중복

수집한 HTML 또는 JSON의 해시를 계산한다.

```
contentHash = SHA-256(normalizedRawContent)
```

이전 콘텐츠 해시와 동일하면 상품 파싱 또는 이력 저장 일부를 생략할 수 있다.

9.3 상품 중복

사이트가 달라도 동일 상품일 수 있으므로 별도의 Canonical Product를 둔다.

중복 판단 기준:

- 브랜드
- 모델 번호
- 제조사 상품 코드
- GTIN, UPC, EAN
- 정규화된 상품명
- 주요 사양

10. 재수집 및 신선도 정책

대상	기본 수집 주기
사용자 검색 중인 상품	즉시
결제 예정 상품	결제 직전
가격	1시간~24시간
재고	10분~6시간
배송 정보	1시간~24시간
상품 설명	7일
리뷰 수	1일
품질 상품	1일~7일

수집 주기는 고정하지 않고 다음 조건을 반영해 계산할 수 있다.

```
recrawlPriority =
사용자 요청 빈도
+ 가격 변경 빈도
+ 상품 인기도
```

- + 현재 데이터 경과 시간
- + 재고 부족 여부

11. Rate Limit과 Politeness

호스트별로 요청 속도를 관리한다.

`RateLimiter Key = hostname`

정책 예시

`shop-a.com:`
`requestsPerSecond: 0.5`
`maxConcurrency: 2`

`shop-b.com:`
`requestsPerSecond: 0.2`
`maxConcurrency: 1`

처리 원칙

- 동일 호스트에 대한 동시 요청 제한
- 요청 사이 최소 간격 적용
- HTTP 429 발생 시 즉시 요청량 감소
- Retry-After 헤더 준수
- robots.txt 결과 캐싱
- 실패가 반복되는 도메인은 Circuit Breaker 적용

12. 실패 처리

실패 유형

`NETWORK_ERROR`
`TIMEOUT`
`HTTP_403`
`HTTP_429`
`HTTP_5XX`
`INVALID_RESPONSE`
`PARSER_ERROR`
`SCHEMA_CHANGED`

VALIDATION_ERROR
STORAGE_ERROR
BROWSER_ERROR

재시도 가능 여부

오류	재시도
네트워크 오류	가능
Timeout	가능
HTTP 429	지연 후 가능
HTTP 500	가능
HTTP 403	자동 반복 금지
Parser 오류	코드 수정 전 불가
Schema 변경	Adapter 수정 전 불가
Validation 오류	원본 확인 필요
저장소 오류	가능

Backoff

1차 재시도: 5초
2차 재시도: 30초
3차 재시도: 5분
최종 실패: Dead Letter Queue

13. 배포 아키텍처

초기 개발 환경

Docker Compose
├ crawler-api
├ crawl-orchestrator
├ http-worker
├ browser-worker
├ parser-worker
├ PostgreSQL
├ Redis
└ RabbitMQ

- └ MinIO
- └ Prometheus
- └ Grafana

운영 확장 환경

- Kubernetes
 - └ API Deployment
 - └ Orchestrator Deployment
 - └ HTTP Worker Deployment
 - └ JSON Worker Deployment
 - └ Browser Worker Deployment
 - └ Parser Worker Deployment
 - └ Redis
 - └ Message Broker
 - └ PostgreSQL
 - └ Object Storage

각 Worker는 무상태로 설계한다.

상태 정보는 다음 외부 저장소에 둔다.

- 작업 상태: PostgreSQL
- 단기 Lock 및 Rate Limit: Redis
- 비동기 작업: RabbitMQ 또는 Kafka
- 원본 HTML-JSON: Object Storage
- 정규화 상품 데이터: PostgreSQL 또는 검색엔진

14. 기술 스택 제안

영역	기술
API	FastAPI 또는 Spring Boot
Orchestrator	Python 또는 Spring Boot
HTTP 수집	HTTPX, aiohttp
HTML 파싱	BeautifulSoup, lxml
Browser	Playwright
메시지 큐	RabbitMQ
관계형 DB	PostgreSQL

영역	기술
캐시·Lock	Redis
원본 저장소	MinIO 또는 S3
검색	OpenSearch 또는 Elasticsearch
모니터링	Prometheus, Grafana
로그	Loki 또는 ELK
배포	Docker Compose → Kubernetes

초기 PoC에서는 Kafka보다 RabbitMQ가 구현과 운영이 단순하다.

수집 이벤트를 장기간 보관하거나 여러 소비자가 동일 이벤트를 재처리해야 하는 규모가 되면 Kafka를 검토한다.

15. 관측성

Metrics

- 크롤링 요청 수
- 사이트별 성공률
- 사이트별 실패율
- 평균 응답 시간
- 파싱 성공률
- 필수 필드 누락률
- HTTP 403·429 비율
- 재시도 횟수
- Queue 적체량
- Worker 처리량
- Browser Worker 메모리
- 상품 데이터 최신성

Log 필드

```
{
  "taskId": "CRAWL_20260725_000001",
  "siteId": "SHOP_A",
  "targetUrl": "https://shop-a.com/products/123456",
  "collectorType": "JSON_API",
  "workerId": "json-worker-03",
  "status": "SUCCESS",
```

```
"durationMs": 532,  
"timestamp": "2026-07-25T17:00:00+09:00"  
}
```

Alert

- 사이트별 성공률 90% 미만
- Parser 오류 급증
- HTTP 403·429 급증
- Queue 대기 작업 기준 초과
- Browser Worker 메모리 기준 초과
- 상품 데이터 적재 실패
- 결제 전 재검증 실패

16. 확장 전략

사이트 추가

새로운 쇼핑몰 추가 시 다음 항목만 구현한다.

1. Site Registry 등록
2. 수집 방식 지정
3. Site Adapter 구현
4. Parser 및 Mapper 구현
5. Fixture 기반 테스트 추가
6. 배포

공통 Orchestrator, Queue, Worker 및 Storage는 수정하지 않는다.

Worker 확장

HTTP 작업 증가
→ HTTP Worker **Replica** 증가

동적 페이지 증가
→ Browser Worker **Replica** 증가

파싱 적체 발생
→ **Parser** Worker **Replica** 증가

콘텐츠 유형 확장

향후 다음 Collector를 플러그인으로 추가할 수 있다.

Image Collector
Review Collector
Shipping Collector
Seller Collector
Specification Collector

17. 테스트 전략

Adapter 테스트

실제 사이트를 매번 호출하지 않고 HTML 및 JSON Fixture로 테스트한다.

```
tests/  
└─ fixtures/  
    └─ shop_a/  
        ├── product_detail.html  
        ├── product_detail.json  
        └─ product_sold_out.json
```

주요 테스트 항목

- 상품명 추출
- 가격 추출
- 품질 판별
- 옵션 파싱
- 페이지네이션
- HTML 구조 일부 누락
- JSON 필드 변경
- 잘못된 가격 문자열
- 중복 데이터
- Timeout
- 429 응답
- Browser 렌더링 실패

Contract Test

사이트 Adapter가 공통 인터페이스를 충족하는지 검증한다.

```
collect()  
parse()  
normalize()  
validate()  
extractNextTargets()
```

18. 주요 설계 결정

ADR-001: 사이트별 Adapter 구조 사용

사이트마다 HTML, API, 인증 및 데이터 구조가 다르기 때문에 사이트별 로직을 공통 코드에서 분리한다.

ADR-002: JSON API 수집 우선

HTML에 동일한 데이터가 있더라도 안정적인 JSON 응답을 사용할 수 있으면 JSON API 방식을 우선한다.

단, 해당 API의 접근 조건과 변경 가능성을 검토해야 한다.

ADR-003: Browser Worker 분리

브라우저 수집은 CPU와 메모리 사용량이 크기 때문에 HTTP Worker와 별도의 서비스로 운영한다.

ADR-004: 원본 데이터와 정규화 데이터 분리

파서 변경과 장애 분석을 위해 원본 HTML 및 JSON을 별도로 보관한다.

ADR-005: 상품과 판매 제안 분리

동일 상품을 여러 판매처에서 비교할 수 있도록 Product와 ProductOffer를 분리한다.

ADR-006: 결제 전 실시간 재검증

저장된 상품 정보는 변경될 수 있으므로 결제 직전에 가격, 재고, 옵션 및 판매 상태를 다시 확인한다.